SAE 1.04

- Création d’une base de données

Bouyssou Martial Romeuf Emma

Introduction

Dans le cadre de la ressource R1.05 (introduction aux bases de données et SQL), ce projet vise à analyser un jeu de données relatives aux ventes de jeux vidéo à travers le monde. Les données, issues du fichier vgsales.csv, fournissent des informations détaillées sur les jeux, leurs plateformes, leurs années de sortie, leurs genres, ainsi que leurs ventes par région géographique (Amérique du Nord, Europe, Japon, etc.).

L’objectif principal de cette étude est de récupérer, structurer et analyser ces données à l’aide du langage SQL. Pour ce faire, le travail s’articule en trois phases :

- La récupération et le nettoyage des données pour garantir leur qualité et leur cohérence.
- La conception d’une base de donnée relationnelle adaptée, suivie de l’import des données nettoyées.
- L’analyse des données via des requêtes SQL, permettant d’extraire des informations pertinentes, qui seront ensuite visualisées sous forme de graphiques.

Cette approche permettra non seulement de comprendre les tendances du marché des jeux vidéo, mais aussi de mettre en pratique les compétences acquises en modélisation de bases de données et en TP de SQL.

Phase 1

Analyse préliminaire du fichier CSV

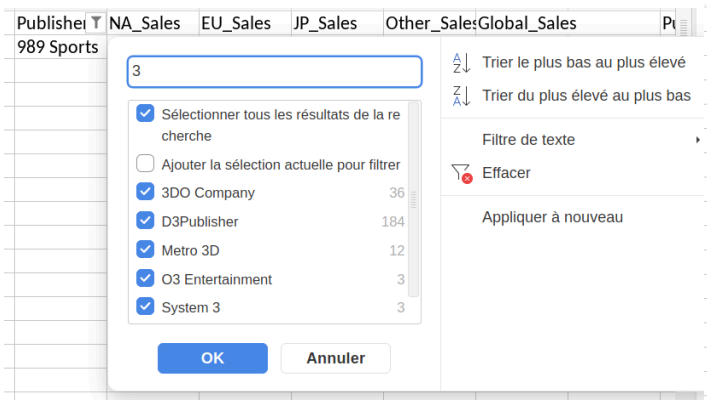
Avant toute manipulation, une analyse approfondie du fichier CSV a été réalisée afin d'identifier les éléments potentiellement contraignants pour son exploitation. Cette étape a permis de relever les points suivants :

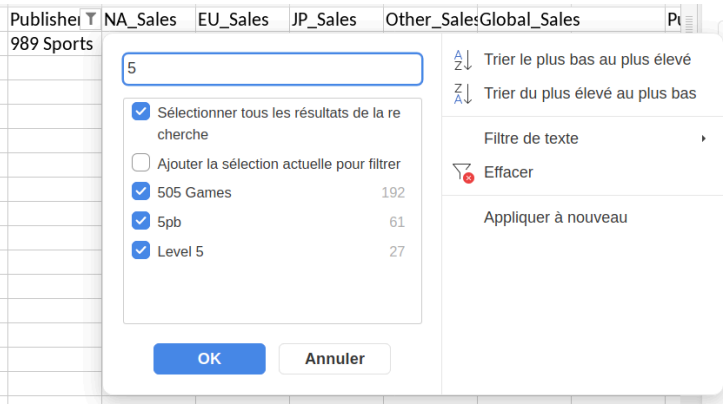
- Valeurs nulles : présence de champs vides ou incomplets, pouvant impacter la qualité des données.
- Format des noms : certains noms de colonnes ou d'entrées étaient composés uniquement de chiffres, ce qui pourrait poser problème pour leur interprétation ou leur traitement automatique.

Suite à cette vérification, des modifications ciblées ont été apportées aux données pour:

- Standardiser les formats, harmonisation des noms de colonnes et des valeurs pour faciliter leur exploitation.
- Corriger les incohérences, remplacement ou suppression des valeurs nulles ou aberrantes, selon leur pertinence.

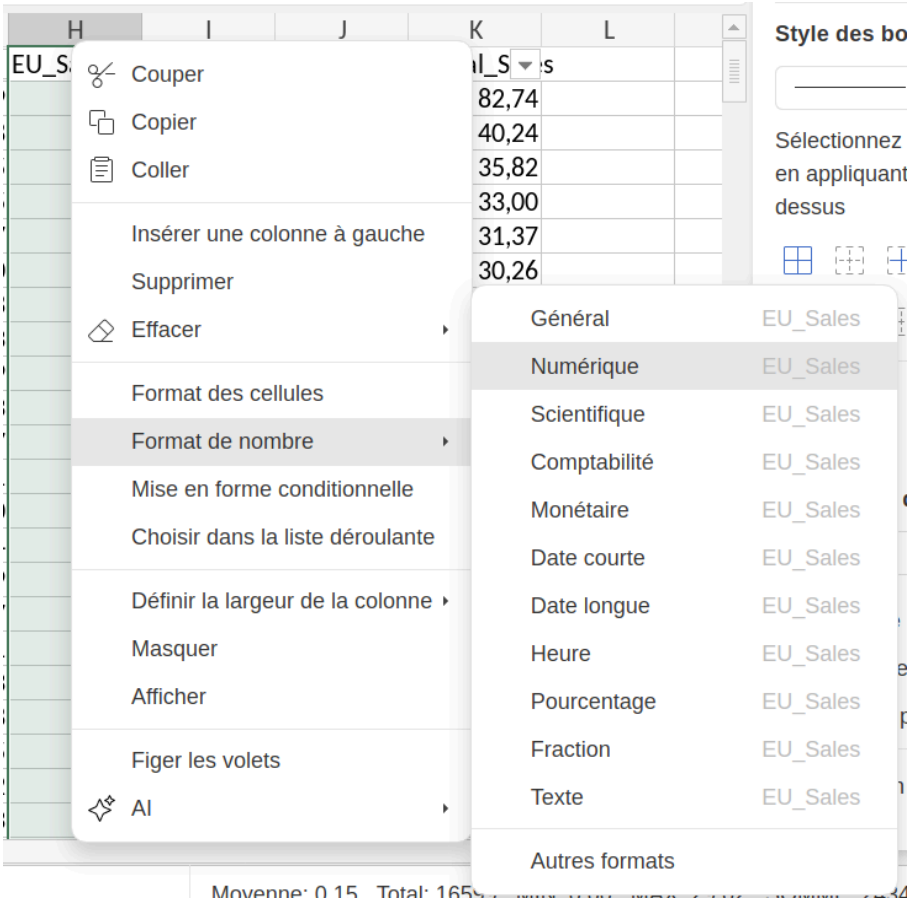
On a donc commencé par vérifier les noms des jeux, des publisher, mais aussi les plateformes et le genre.







Une fois le tout vérifié, nous avons donc choisi de prendre aussi le temps de vérifier comment été géré les nombres présents dans les Sales afin de ré adapter au mieux le tout pour le traitement des données.



Phase 2

L'objectif de cette seconde phase est de concevoir un modèle de données relationnel à partir des données brut analysées dans la phase 1, puis de mettre en place physiquement la base de données.
Ainsi, cette phase comprend:

- La définition d'un schéma relationnel cohérent, limitant au maximum les redondances;
- La création des tables et des contraintes sur celle-ci;
- L'alimentation des nouvelles tables à partir de la table contenant les données brutes (vgsales).

Cette étape constitue une transition entre l'analyse des données et leur exploitation analytique en phase 3.

Avant la création du modèle relationnel, certaines corrections ont été apportées à la table pour garantir la cohérence des données et la viabilité dans le temps.
En particulier, la colonne Year contenait la valeur textuelle 'N/A', incompatible avec un type numérique. Cette valeur a donc été remplacée par NULL, puis la colonne a pu être convertie en SMALLINT.
De plus, les colonnes représentant les ventes par zone géographique ont été converties en type numériques (DECIMAL) et rendues non nulles.

```
UPDATE dbo.vgsales SET [Year] = NULL WHERE [Year] = 'N/A';

ALTER TABLE dbo.vgsales
ALTER COLUMN [Year] SMALLINT;

ALTER TABLE dbo.vgsales
ALTER COLUMN NA_Sales DECIMAL(6,2) NOT NULL;

ALTER TABLE dbo.vgsales
ALTER COLUMN EU_Sales DECIMAL(6,2) NOT NULL;

ALTER TABLE dbo.vgsales
ALTER COLUMN JP_Sales DECIMAL(6,2) NOT NULL;
```

```sql
ALTER TABLE dbo.vgsales
ALTER COLUMN Other_Sales DECIMAL(6,2) NOT NULL;

ALTER TABLE dbo.vgsales
ALTER COLUMN Global_Sales DECIMAL(6,2) NOT NULL;

SELECT * FROM dbo.vgsales
```

Ces transformations permettent d'assurer la qualité des données avant leur intégration dans le modèle relationnel final.

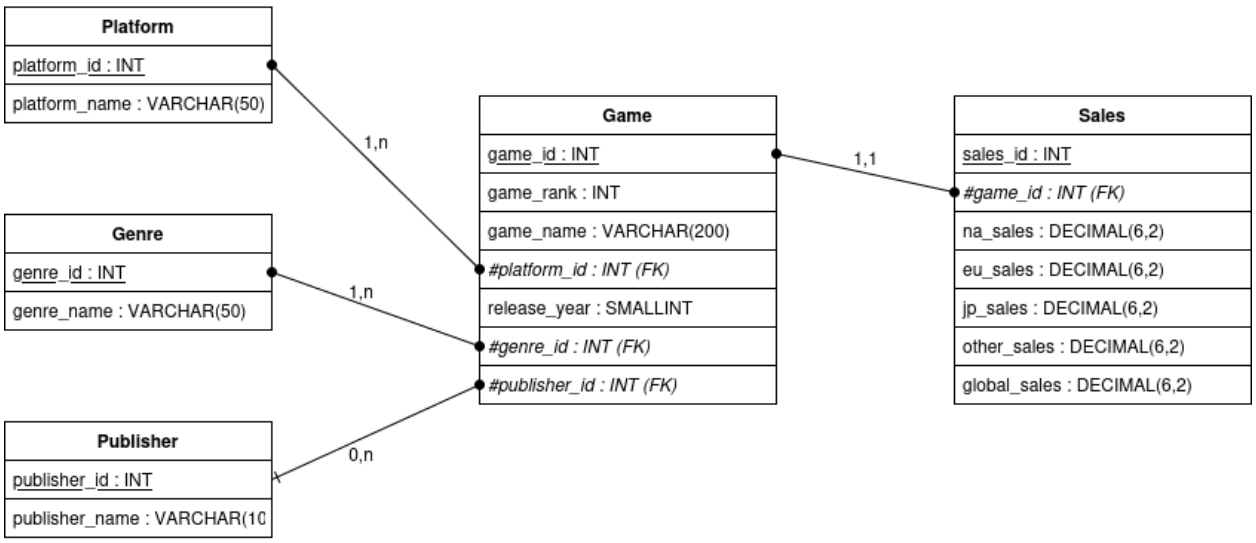
À partir de l'analyse réalisée en phase 1, il est apparu que certaines informations étaient redondantes dans la table initiale, notamment les plateformes, genres et éditeurs.
De la sorte, le modèle a été décomposé en plusieurs tables distinctes.
Le modèle est donc composé des entités suivantes:

- Platform: liste des plateformes de jeux;
- Genre: liste des genres;
- Publisher: liste des éditeurs;
- Game: informations générales sur les jeux;
- Sales: données de ventes par zone géographique.

Chaque jeu est associé à une plateforme, un genre, un éditeur (optionnel) et une ligne de ventes.

Ainsi nous obtenons le modèle relationnel suivant:

Schéma Relationnel - Base VGSales



La séparation des entités Platform, Genre et Publisher permet d'éviter les répétitions de chaînes de caractères et facilite les jointure. De plus, la table Game centralise les informations descriptives d'un jeu. De même, la table Sales est séparée afin de regrouper toutes les mesures numériques.
Les clés primaires sont les identifiants tandis que les clés étrangères référence vers les autres tables. Des contraintes CHECK assurent la validité des données (années, vente positives, etc...).

Ainsi grâce au script suivant nous avons pu créer l'ensemble des tables du modèle relationnel, ainsi que leurs contraintes.

```
CREATE TABLE dbo.Platform (
    platform_id INT IDENTITY(1,1) PRIMARY KEY,
    platform_name VARCHAR(50) NOT NULL UNIQUE
);
```

```sql
CREATE TABLE dbo.Genre (
    genre_id INT IDENTITY(1,1) PRIMARY KEY,
    genre_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE dbo.Publisher (
    publisher_id INT IDENTITY(1,1) PRIMARY KEY,
    publisher_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE dbo.Game (
    game_id INT IDENTITY(1,1) PRIMARY KEY,
    game_rank INT NOT NULL,
    game_name VARCHAR(200) NOT NULL,
    platform_id INT NOT NULL,
    release_year SMALLINT NULL,
    genre_id INT NOT NULL,
    publisher_id INT NULL,

    CONSTRAINT FK_Game_Platform FOREIGN KEY (platform_id)
        REFERENCES dbo.Platform(platform_id),
    CONSTRAINT FK_Game_Genre FOREIGN KEY (genre_id)
        REFERENCES dbo.Genre(genre_id),
    CONSTRAINT FK_Game_Publisher FOREIGN KEY (publisher_id)
        REFERENCES dbo.Publisher(publisher_id),

    CONSTRAINT CK_Game_Year CHECK (release_year IS NULL OR (release_year >=
1970 AND release_year <= 2100))
);

CREATE TABLE dbo.Sales (
    sales_id INT IDENTITY(1,1) PRIMARY KEY,
    game_id INT NOT NULL UNIQUE,
    na_sales DECIMAL(6,2) NOT NULL DEFAULT 0,
    eu_sales DECIMAL(6,2) NOT NULL DEFAULT 0,
    jp_sales DECIMAL(6,2) NOT NULL DEFAULT 0,
    other_sales DECIMAL(6,2) NOT NULL DEFAULT 0,
    global_sales DECIMAL(6,2) NOT NULL DEFAULT 0,

    CONSTRAINT FK_Sales_Game FOREIGN KEY (game_id)
        REFERENCES dbo.Game(game_id) ON DELETE CASCADE,
```

```
    CONSTRAINT CK_Sales_NA CHECK (na_sales >= 0),
    CONSTRAINT CK_Sales_EU CHECK (eu_sales >= 0),
    CONSTRAINT CK_Sales_JP CHECK (jp_sales >= 0),
    CONSTRAINT CK_Sales_Other CHECK (other_sales >= 0),
    CONSTRAINT CK_Sales_Global CHECK (global_sales >= 0)
);
```

Une fois les tables créées, elles ont été alimentées à partir de la table vgsales (phase 1) en utilisant des requêtes INSERT INTO ... SELECT:

```
INSERT INTO dbo.Platform (platform_name)
SELECT DISTINCT Platform
FROM dbo.vgsales
WHERE Platform IS NOT NULL
ORDER BY Platform;

INSERT INTO dbo.Genre (genre_name)
SELECT DISTINCT Genre
FROM dbo.vgsales
WHERE Genre IS NOT NULL
ORDER BY Genre;

INSERT INTO dbo.Publisher (publisher_name)
SELECT DISTINCT Publisher
FROM dbo.vgsales
WHERE Publisher IS NOT NULL
ORDER BY Publisher;

INSERT INTO dbo.Game (game_rank, game_name, platform_id, release_year,
genre_id, publisher_id)
SELECT
    v.[Rank],
    v.[Name],
    p.platform_id,
    v.[Year],
    g.genre_id,
    pub.publisher_id
FROM dbo.vgsales v
INNER JOIN dbo.Platform p ON v.Platform = p.platform_name
INNER JOIN dbo.Genre g ON v.Genre = g.genre_name
LEFT JOIN dbo.Publisher pub ON v.Publisher = pub.publisher_name;
```

```
INSERT INTO dbo.Sales (game_id, na_sales, eu_sales, jp_sales, other_sales,
global_sales)
SELECT
    gm.game_id,
    v.NA_Sales,
    v.EU_Sales,
    v.JP_Sales,
    v.Other_Sales,
    v.Global_Sales
FROM dbo.vgsales v
INNER JOIN dbo.Platform p ON v.Platform = p.platform_name
INNER JOIN dbo.Genre g ON v.Genre = g.genre_name
LEFT JOIN dbo.Publisher pub ON v.Publisher = pub.publisher_name
INNER JOIN dbo.Game gm ON gm.game_rank = v.[Rank]
    AND gm.game_name = v.[Name]
    AND gm.platform_id = p.platform_id;
```

Pour conclure, cette phase à permis de concevoir et implémenter un modèle relationnel robuste et viable dans le temps, garantissant l'intégrité de nos données. Les données issues du fichier CSV ont été correctement nettoyées, structurées et intégrées dans la base finale. Ce modèle est, à ce point, une base solide pour pouvoir exécuter les requêtes de la phase 3, dédiée à l'analyse des données et à la production de graphiques à l'aide de différentes requêtes SQL.

Phase 3

Cette troisième phase a pour objectif de mettre en œuvre des requêtes SQL sur le modèle de données conçu, afin d'extraire et d'analyser les informations pertinentes. Les résultats obtenus sont ensuite restitués sous forme de graphiques, grâce à l'outil SQLPad disponible sur les postes informatiques de l'IUT.

Il nous a donc été demandé une rédaction et exécution des requêtes.

Des requêtes SQL ont été élaborées pour interroger la base de données et répondre aux besoins spécifiques de l'analyse. Ces requêtes permettent de :

- Filtrer et agréger les données selon des critères définis ;

- Calculer des indicateurs clés (moyennes, totaux, tendances, etc.) ;

- Croiser les informations pour en dégager des insights.

Une fois cela réalisé, il fallait visualiser les graphiques associés aux différentes requêtes. Pour cela, l'outil SQLPad a été utilisé pour transformer les résultats des requêtes en représentations graphiques (histogrammes, camemberts, courbes, etc.). Ces visualisations facilitent l'interprétation des données et la prise de décision.

On nous a demandé d'afficher la liste des plateformes ayant plus de 20 M de jeux vendus en Europe depuis 2010. Pour ce faire, nous avons réalisé la requête SQL suivante :

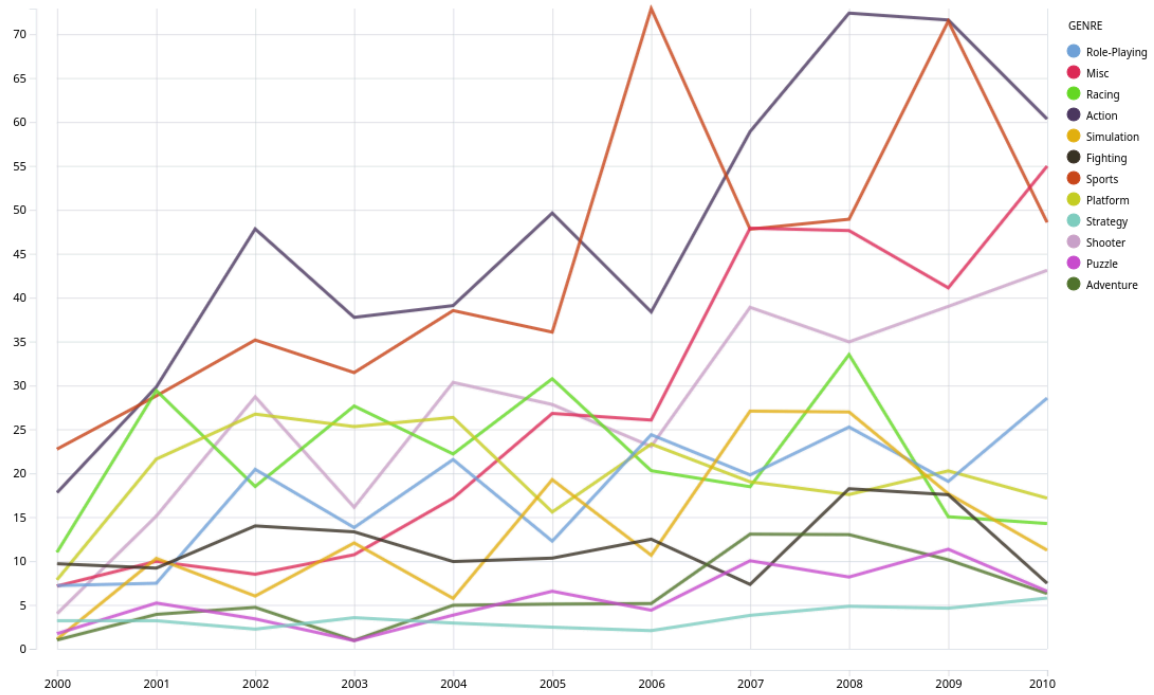
```sql
/* EXERCICE 1 */
SELECT p.platform_name
FROM dbo.Game g
INNER JOIN dbo.Platform p ON g.platform_id = p.platform_id
INNER JOIN dbo.Sales s ON g.game_id = s.game_id
WHERE g.release_year >= 2010
GROUP BY p.platform_name
HAVING SUM(s.eu_sales) > 20
```

Pour répondre à la demande d'affichage des éditeurs de jeux PS 4 ayant dépassé les 2 millions de ventes, toute années confondues, la requête suivante a été élaborée. Les résultats sont triés par ordre décroissant du nombre total de jeux vendus :

```sql
/* EXERCICE 2 */
SELECT pub.publisher_name
FROM dbo.Game g
INNER JOIN dbo.Platform p ON g.platform_id = p.platform_id
INNER JOIN dbo.Publisher pub ON g.publisher_id = pub.publisher_id
INNER JOIN dbo.Sales s ON g.game_id = s.game_id
WHERE p.platform_name = 'PS4' AND s.global_sales > 2
ORDER BY s.global_sales DESC
```

Pour étudier l'évolution du nombre de ventes de jeux par genre en Amérique du Nord sur la période 2000 à 2010, la requête SQL suivante a été conçue. Les résultats permettront de construire un graphique en courbes illustrant les tendances par genre au fil des années.

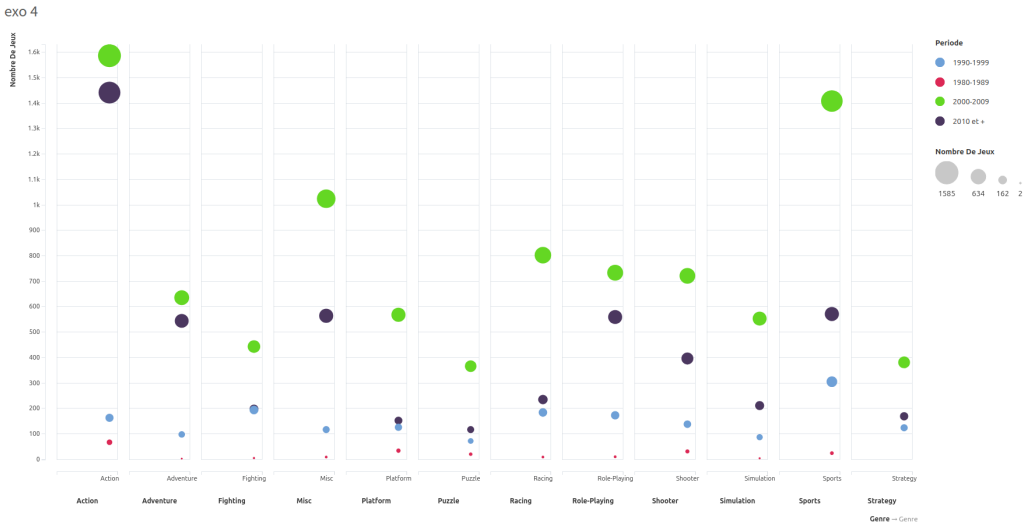
```sql
/* EXERCICE 3 */
SELECT g.release_year, ge.genre_name, SUM(s.na_sales) as Sales
FROM dbo.Game g
INNER JOIN dbo.Genre ge ON g.genre_id = ge.genre_id
INNER JOIN dbo.Sales s ON g.game_id = s.game_id
WHERE g.release_year BETWEEN 2000 AND 2010
GROUP BY g.release_year, ge.genre_name
```



Cette requête nous renvoie le graphique suivant pour mieux comprendre l'évolution du nombre de ventes de jeux par genre en Amérique du Nord sur la période 2000 à 2010.

Pour étudier l'évolution du nombre de jeux édités en fonction du genre, répartis en quatre tranches décennales (1980–1989, 1990–1999, 2000–2009, 2010 et plus), la requête SQL suivante a été élaborée. Les résultats permettront de construire un graphique en barres empilées ou un histogramme groupé pour visualiser les tendances par genre au fil des décennies.

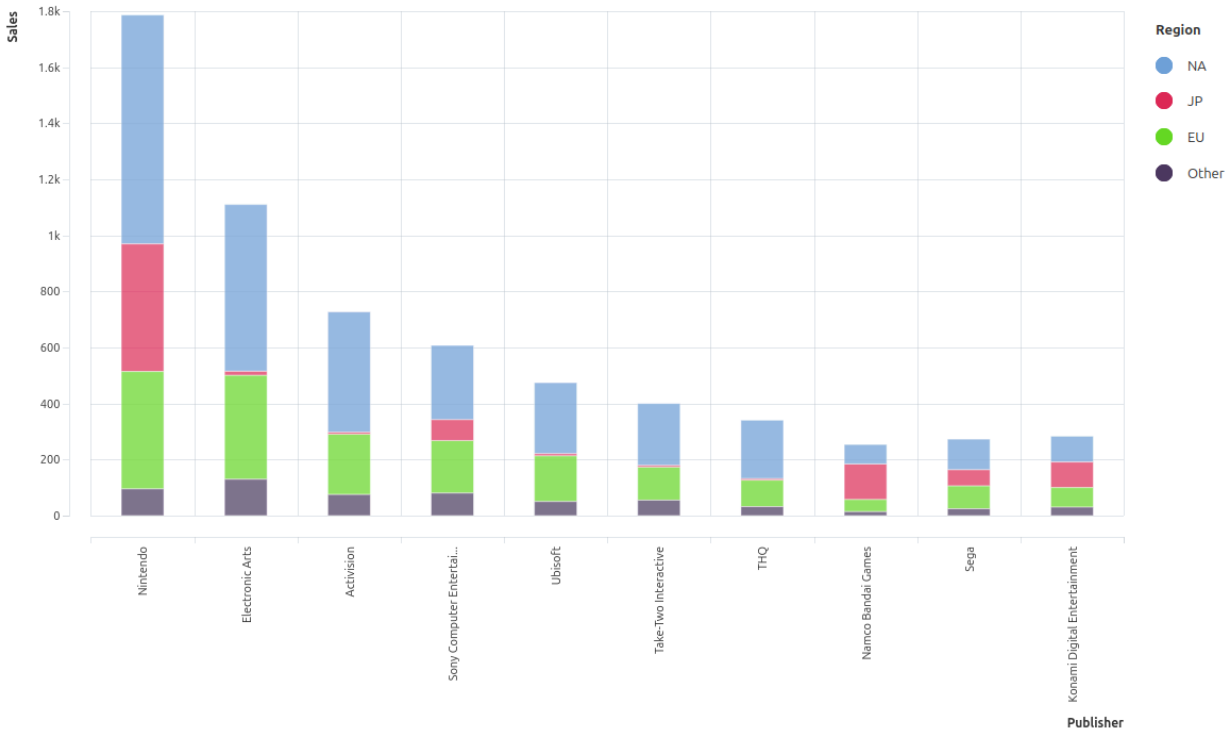
```sql
/* EXERCICE 4 */
SELECT
    CASE
        WHEN g.release_year BETWEEN 1980 AND 1989 THEN '1980-1989'
        WHEN g.release_year BETWEEN 1990 AND 1999 THEN '1990-1999'
        WHEN g.release_year BETWEEN 2000 AND 2009 THEN '2000-2009'
        WHEN g.release_year >= 2010 THEN '2010 et +'
        ELSE 'Erreur'
    END AS Periode,
    ge.genre_name,
    COUNT(*) as 'Nombre de Jeux'
FROM dbo.Game g
INNER JOIN dbo.Genre ge ON g.genre_id = ge.genre_id
WHERE g.release_year >= 1980
GROUP BY
    CASE
        WHEN g.release_year BETWEEN 1980 AND 1989 THEN '1980-1989'
        WHEN g.release_year BETWEEN 1990 AND 1999 THEN '1990-1999'
        WHEN g.release_year BETWEEN 2000 AND 2009 THEN '2000-2009'
        WHEN g.release_year >= 2010 THEN '2010 et +'
        ELSE 'Erreur'
    END,
    ge.genre_name
ORDER BY Periode ASC
```



Avec ce graphique, on peut donc remarquer que l'évolution du nombre de jeux édités en fonction du genre, répartis en quatre tranches décennales (1980–1989, 1990–1999, 2000–2009, 2010 et plus).

Afin d'évaluer l'impact géographique des ventes des 10 éditeurs les plus influents du marché mondial, cette requête SQL extrait et compare leurs performances commerciales selon quatre régions clé, Amérique du Nord, Europe, Japon et Autres marchés. Les données sont agrégées pour chaque éditeur, puis triées afin de mettre en avant les 40 meilleures performances régionales, offrant ainsi une vision ciblée des dynamiques commerciales à l'échelle internationale.

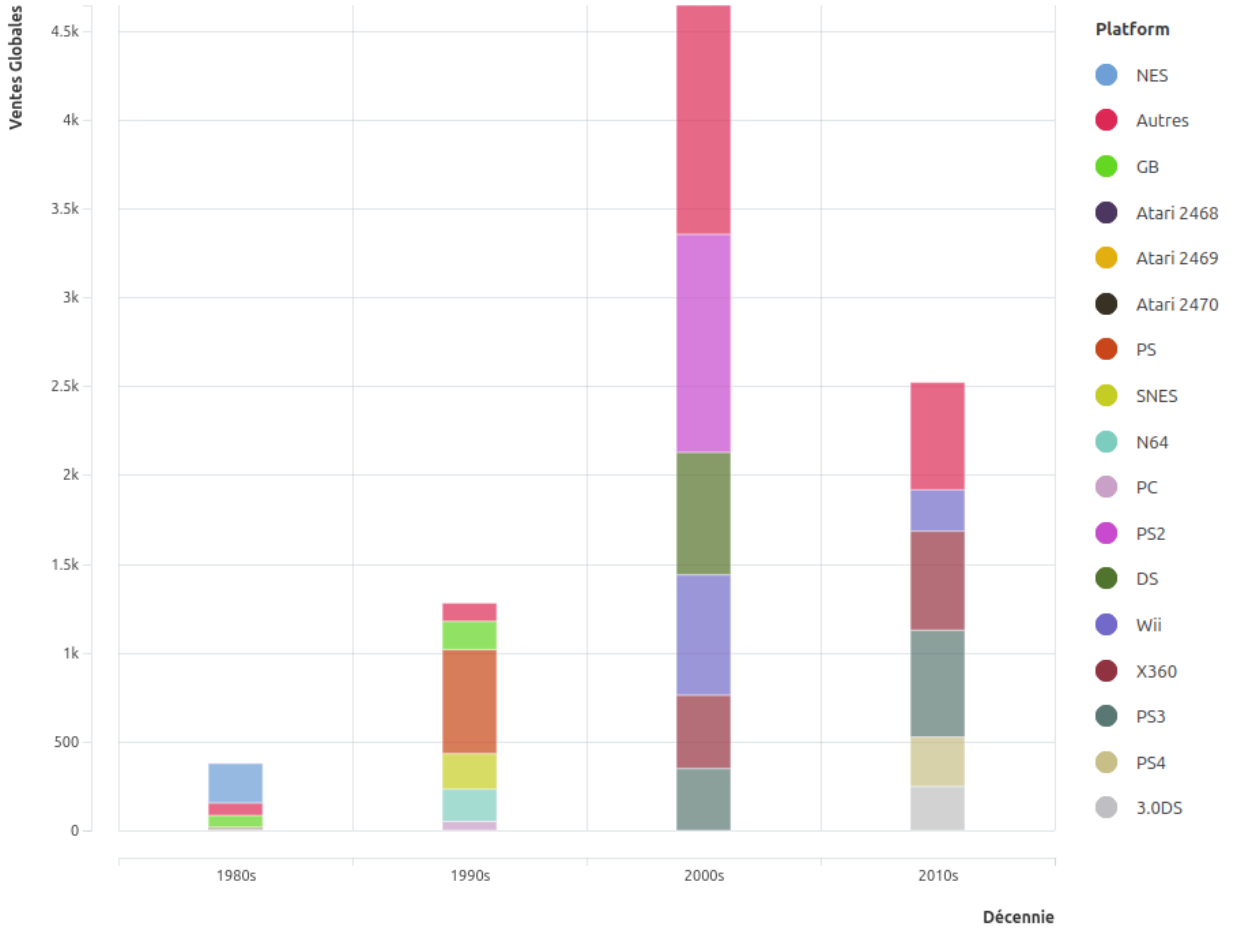
```sql
/* EXERCICE 5 - 1 */
SELECT TOP 40
    publisher_name,
    Region,
    ROUND(Sales, 2) as Sales
FROM (
    SELECT pub.publisher_name, SUM(s.na_sales) as Sales, 'NA' as Region
    FROM dbo.Game g
    INNER JOIN dbo.Publisher pub ON g.publisher_id = pub.publisher_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY pub.publisher_name
    UNION ALL
    SELECT pub.publisher_name, SUM(s.eu_sales), 'EU'
    FROM dbo.Game g
    INNER JOIN dbo.Publisher pub ON g.publisher_id = pub.publisher_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY pub.publisher_name
    UNION ALL
    SELECT pub.publisher_name, SUM(s.jp_sales), 'JP'
    FROM dbo.Game g
    INNER JOIN dbo.Publisher pub ON g.publisher_id = pub.publisher_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY pub.publisher_name
    UNION ALL
    SELECT pub.publisher_name, SUM(s.other_sales), 'Other'
    FROM dbo.Game g
    INNER JOIN dbo.Publisher pub ON g.publisher_id = pub.publisher_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY pub.publisher_name
) AS RegionalSales
WHERE publisher_name IN (
    SELECT TOP 10 pub.publisher_name
    FROM dbo.Game g
    INNER JOIN dbo.Publisher pub ON g.publisher_id = pub.publisher_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY pub.publisher_name
    ORDER BY SUM(s.global_sales) DESC
)
ORDER BY Sales DESC
```



Ce graphique illustre la répartition des ventes des 10 éditeurs les plus performants au niveau mondial, en comparant leur succès commercial dans quatre régions clé : Amérique du Nord, Europe, Japon et Autres marchés.

Cette requête permet d'analyser l'évolution des ventes globales par plateforme sur quatre décennies (1980 s, 1990s, 2000s, 2010s), en identifiant les 5 plateformes les plus performantes par décennie et en regroupant les autres sous la catégorie "Autres", afin de visualiser les tendances dominantes du marché du jeu vidéo au fil du temps.

```sql
/* EXERCICE 5 - 2 */
WITH DecennieSales AS (
    SELECT
        CASE
            WHEN g.release_year BETWEEN 1980 AND 1989 THEN '1980s'
            WHEN g.release_year BETWEEN 1990 AND 1999 THEN '1990s'
            WHEN g.release_year BETWEEN 2000 AND 2009 THEN '2000s'
            WHEN g.release_year >= 2010 THEN '2010s'
        END AS Décennie,
        p.platform_name,
        SUM(s.global_sales) as Ventes
    FROM dbo.Game g
    INNER JOIN dbo.Platform p ON g.platform_id = p.platform_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    WHERE g.release_year >= 1980
    GROUP BY
        CASE
            WHEN g.release_year BETWEEN 1980 AND 1989 THEN '1980s'
            WHEN g.release_year BETWEEN 1990 AND 1999 THEN '1990s'
            WHEN g.release_year BETWEEN 2000 AND 2009 THEN '2000s'
            WHEN g.release_year >= 2010 THEN '2010s'
        END,
        p.platform_name
),
Top5PerDecennie AS (
    SELECT
        Décennie,
        platform_name,
        Ventes,
        ROW_NUMBER() OVER (PARTITION BY Décennie ORDER BY Ventes DESC) AS
Rang
    FROM DecennieSales
)
SELECT
    Décennie,
    CASE WHEN Rang <= 5 THEN platform_name ELSE 'Autres' END AS Platform,
    ROUND(SUM(Ventes), 2) as 'Ventes Globales'
FROM Top5PerDecennie
GROUP BY Décennie, CASE WHEN Rang <= 5 THEN platform_name ELSE 'Autres' END
ORDER BY Décennie, SUM(Ventes) DESC
```



Ce graphique présente l'évolution des ventes globales par plateforme depuis les années 1980, en mettant en avant les 5 plateformes les plus performantes par décennie, afin d'illustrer les tendances majeures et les changements de leadership dans l'industrie du jeu vidéo.

Cette requête permet d'analyser les ventes totales par genre de jeu selon trois régions (Amérique du Nord, Europe et Japon), afin de comparer la popularité des différents genres sur chaque marché.

```sql
/* EXERCICE 5 - 3 */
SELECT
    genre_name,
    Région,
    ROUND(Ventes, 2) as Ventes
FROM (
    SELECT ge.genre_name, 'NA' as Région, SUM(s.na_sales) as Ventes
    FROM dbo.Game g
    INNER JOIN dbo.Genre ge ON g.genre_id = ge.genre_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY ge.genre_name
    UNION ALL
    SELECT ge.genre_name, 'EU', SUM(s.eu_sales)
    FROM dbo.Game g
    INNER JOIN dbo.Genre ge ON g.genre_id = ge.genre_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY ge.genre_name
    UNION ALL
    SELECT ge.genre_name, 'JP', SUM(s.jp_sales)
    FROM dbo.Game g
    INNER JOIN dbo.Genre ge ON g.genre_id = ge.genre_id
    INNER JOIN dbo.Sales s ON g.game_id = s.game_id
    GROUP BY ge.genre_name
) AS RegionalGenre
ORDER BY genre_name, Région
```

Cette requête nous renvoie un graphique qui compare les ventes totales par genre de jeu entre l'Amérique du Nord, l'Europe et le Japon, révélant les préférences régionales et les disparités culturelles dans la consommation des jeux vidéo.

Conclusion

Ce projet, nous a permis de vivre de manière tangible tout le processus de gestion des données, depuis le fichier brut jusqu'à l'élaboration de tableaux de bord pertinents.

Cette mission, structurée en trois phases principales, nous a permis de :

- Assurer la qualité des données (Phase 1) : en examinant le fichier vgsales.csv, nous avons vite compris l'importance d'effectuer un nettoyage. Le traitement des données absentes et l'uniformisation des formats ne sont pas uniquement des opérations de gestion, elles forment la base indispensable pour assurer la crédibilité des informations que nous extrairons par la suite.
- Organiser l'information de manière structurée (Phase 2) : le passage d'un fichier plat massif à une base de données relationnelle bien structurée (comprenant des tables séparées pour les jeux, les plateformes, les genres, etc.) a tout transformé. Nous avons supprimé les redondances, optimisé l'espace de stockage et constitué une base de données beaucoup plus aisée à entretenir et à développer.
- Exploiter les données (Phase 3) : grâce à l'utilisation de requêtes SQL avancées (intégrant des jointures, des agrégations et même des classements) via SQLPad, nous avons converti des requêtes SQL en graphiques et indicateurs explicites. On a eu l'occasion de voir, par exemple, comment les goûts des joueurs ont changé au fil des décennies, ou d'établir une comparaison entre les marchés américain, européen et japonais.

En fin de compte, ce projet nous a permis de renforcer nos aptitudes en SQL et en modélisation, en nous confrontant aux véritables interrogations que l'on rencontre dans la vie professionnelle.